

ASSIGNMENT - 3.1

2303A51059

Batch - 29

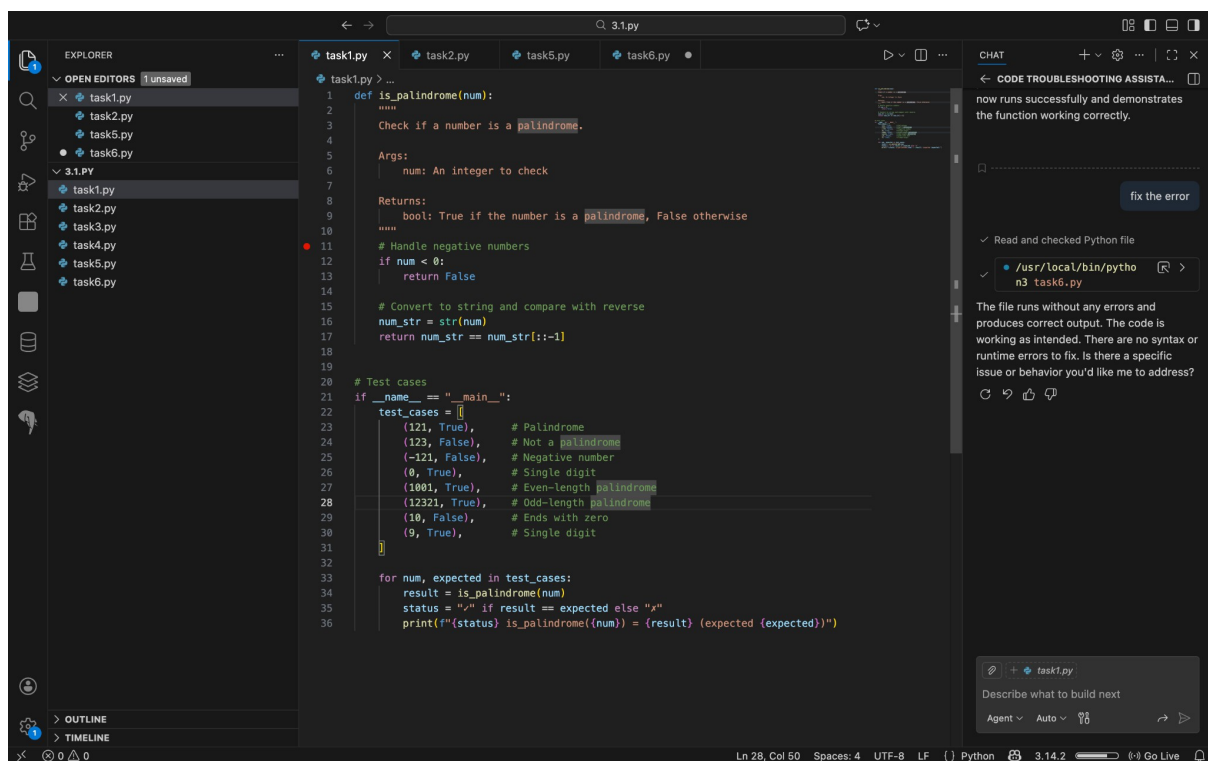
Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

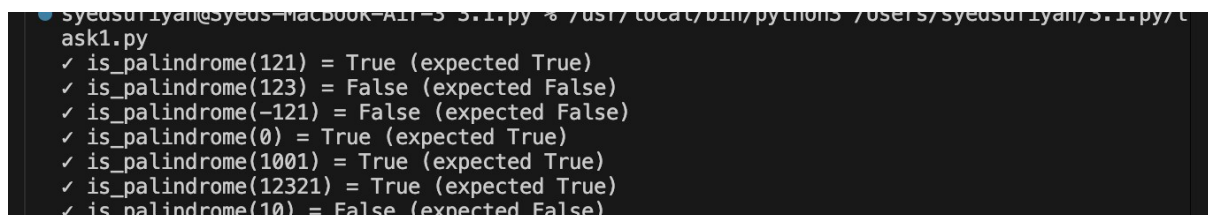
- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Code :



```
1 def is_palindrome(num):
2     """
3     Check if a number is a palindrome.
4
5     Args:
6         num: An integer to check
7
8     Returns:
9         bool: True if the number is a palindrome, False otherwise
10    """
11    # Handle negative numbers
12    if num < 0:
13        return False
14
15    # Convert to string and compare with reverse
16    num_str = str(num)
17    return num_str == num_str[::-1]
18
19
20 # Test cases
21 if __name__ == "__main__":
22     test_cases = [
23         (121, True), # Palindrome
24         (123, False), # Not a Palindrome
25         (-121, False), # Negative number
26         (0, True), # Single digit
27         (1001, True), # Even-length Palindrome
28         (12321, True), # Odd-length Palindrome
29         (10, False), # Ends with zero
30         (9, True), # Single digit
31     ]
32
33     for num, expected in test_cases:
34         result = is_palindrome(num)
35         status = "✓" if result == expected else "✗"
36         print(f"{status} is_palindrome({num}) = {result} (expected {expected})")
```

Output :



```
syedsuliyah@Syeds-MacBook-Air:~/S/3.1.py % ./usr/local/bin/python3 /Users/syedsuliyah/3.1.py/task1.py
✓ is_palindrome(121) = True (expected True)
✓ is_palindrome(123) = False (expected False)
✓ is_palindrome(-121) = False (expected False)
✓ is_palindrome(0) = True (expected True)
✓ is_palindrome(1001) = True (expected True)
✓ is_palindrome(12321) = True (expected True)
✓ is_palindrome(10) = False (expected False)
```

Observation:

The program correctly checks whether a given number is a palindrome by converting it to a string and comparing it with its reverse. It properly handles negative numbers by returning `False` and works accurately for single-digit numbers, even-length and odd-length palindromes, and non-palindromic values. The test cases confirm that the logic handles common edge cases such as numbers ending with zero and negative inputs, demonstrating reliable and correct behavior.

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

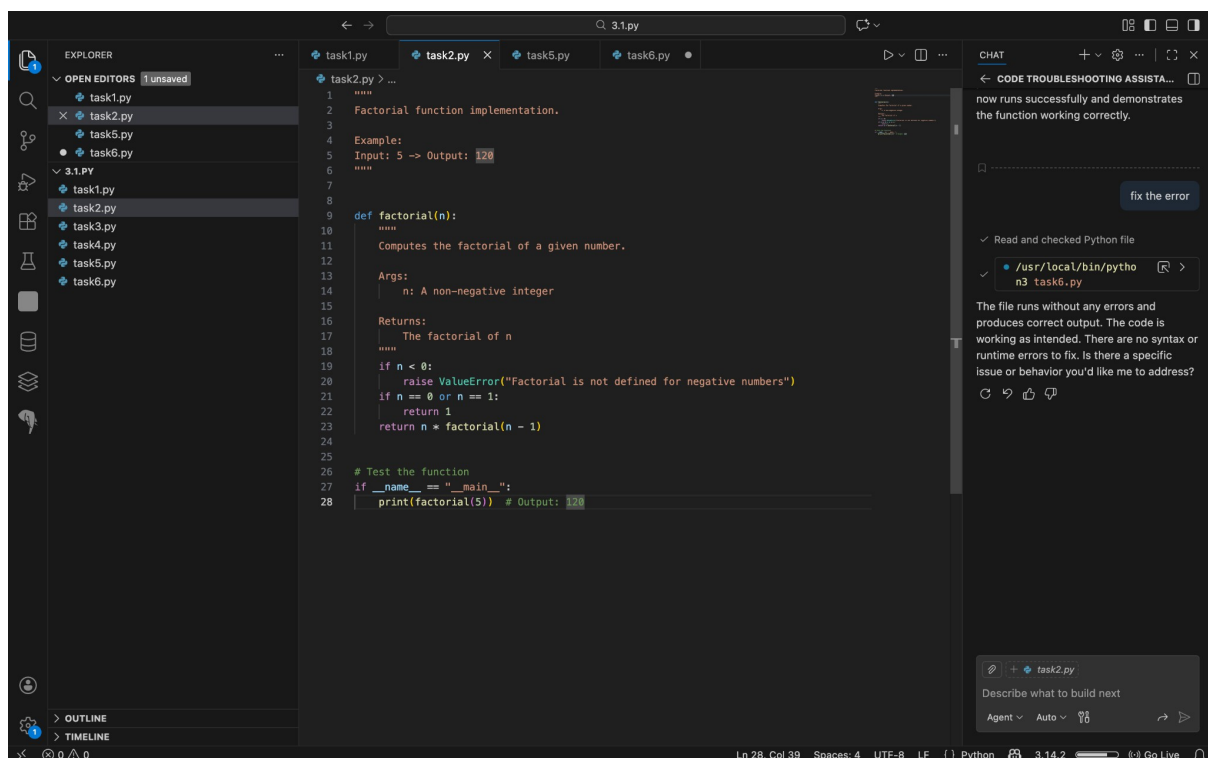
Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

Code :



The screenshot shows a VS Code editor with a file explorer on the left containing several `task*.py` files. The main editor window displays `task2.py` with the following Python code:

```
1 """
2 Factorial function implementation.
3
4 Example:
5 Input: 5 -> Output: 120
6 """
7
8
9 def factorial(n):
10     """
11     Computes the factorial of a given number.
12
13     Args:
14         n: A non-negative integer
15
16     Returns:
17         The factorial of n
18     """
19     if n < 0:
20         raise ValueError("Factorial is not defined for negative numbers")
21     if n == 0 or n == 1:
22         return 1
23     return n * factorial(n - 1)
24
25
26 # Test the function
27 if __name__ == "__main__":
28     print(factorial(5)) # Output: 120
```

On the right side, there is a chat window titled "CODE TROUBLESHOOTING ASSISTANT". It contains the following text:

now runs successfully and demonstrates the function working correctly.

fix the error

✓ Read and checked Python file

✓ `/usr/local/bin/python3 task6.py`

The file runs without any errors and produces correct output. The code is working as intended. There are no syntax or runtime errors to fix. Is there a specific issue or behavior you'd like me to address?

At the bottom of the chat window, there is a section for "task2.py" with the prompt "Describe what to build next" and buttons for "Agent", "Auto", and a play button.

Output :

```
usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task1.py
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.p
sk1.py
is_palindrome(121) = True (expected True)
is_palindrome(123) = False (expected False)
is_palindrome(-121) = False (expected False)
is_palindrome(0) = True (expected True)
is_palindrome(1001) = True (expected True)
is_palindrome(12321) = True (expected True)
is_palindrome(10) = False (expected False)
is_palindrome(9) = True (expected True)
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.p
sk2.py
20
syedsufiyan@Syeds-MacBook-Air-3 3.1.py %
```

Observation:

The program correctly computes the factorial of a given number using a recursive approach. It handles base cases for 0 and 1 by returning 1 and raises an appropriate error for negative inputs, ensuring input validity. The recursive logic breaks the problem into smaller subproblems, and the test case confirms correct output for valid input, demonstrating proper use of recursion and error handling.

Task-3 :

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

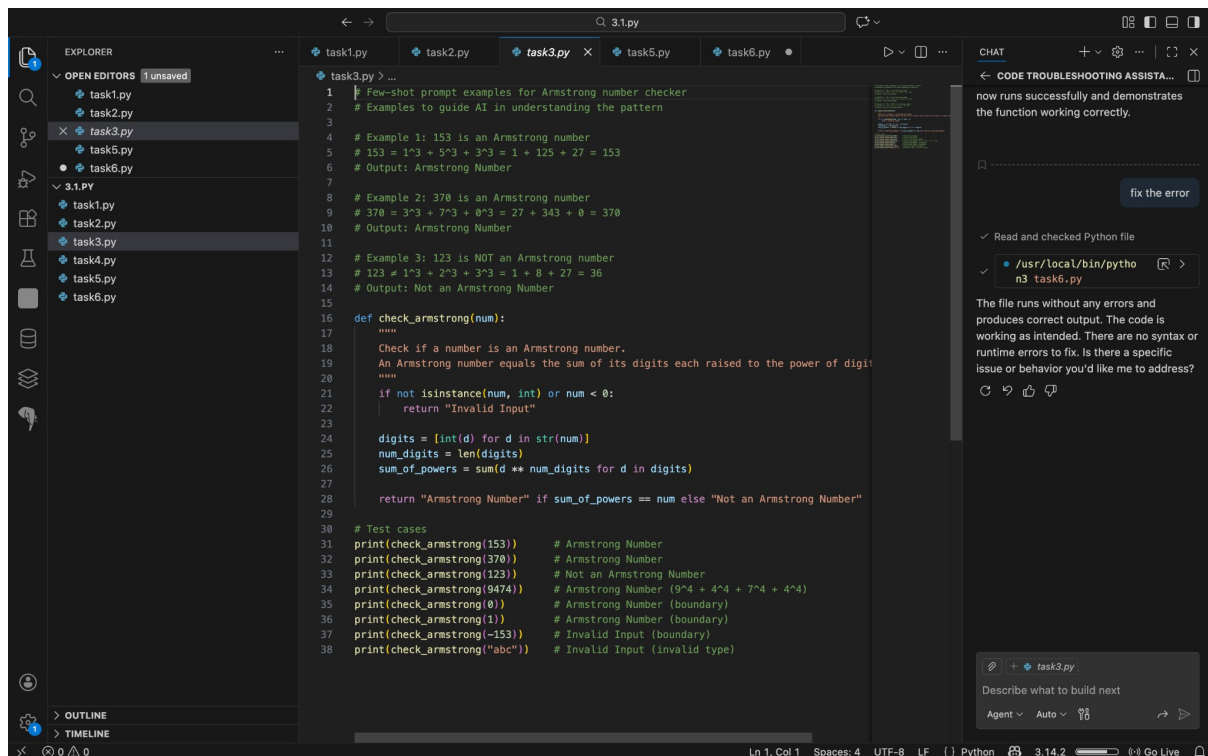
Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

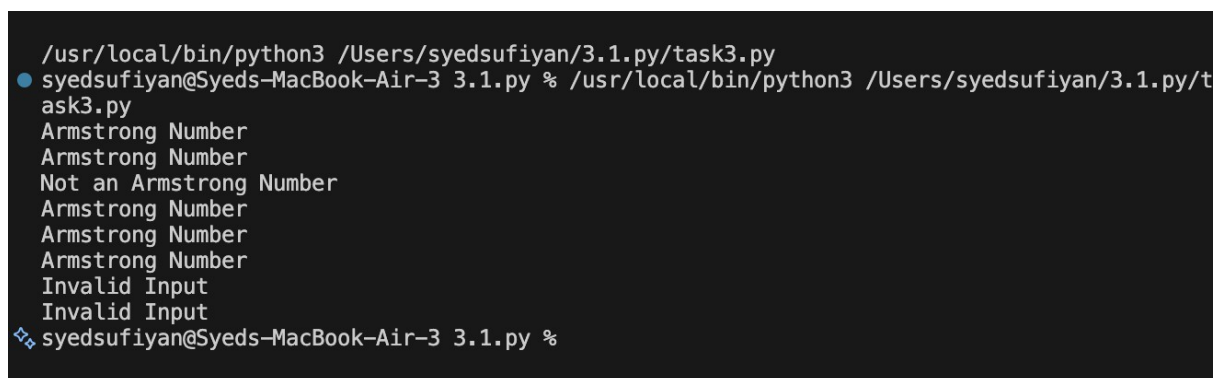
- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Code :



```
1 # Few-shot prompt examples for Armstrong number checker
2 # Examples to guide AI in understanding the pattern
3
4 # Example 1: 153 is an Armstrong number
5 # 153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153
6 # Output: Armstrong Number
7
8 # Example 2: 370 is an Armstrong number
9 # 370 = 3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370
10 # Output: Armstrong Number
11
12 # Example 3: 123 is NOT an Armstrong number
13 # 123 = 1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36
14 # Output: Not an Armstrong Number
15
16 def check_armstrong(num):
17     """
18     Check if a number is an Armstrong number.
19     An Armstrong number equals the sum of its digits each raised to the power of digit
20     """
21     if not isinstance(num, int) or num < 0:
22         return "Invalid Input"
23
24     digits = [int(d) for d in str(num)]
25     num_digits = len(digits)
26     sum_of_powers = sum(d ** num_digits for d in digits)
27
28     return "Armstrong Number" if sum_of_powers == num else "Not an Armstrong Number"
29
30 # Test cases
31 print(check_armstrong(153)) # Armstrong Number
32 print(check_armstrong(370)) # Armstrong Number
33 print(check_armstrong(123)) # Not an Armstrong Number
34 print(check_armstrong(9474)) # Armstrong Number (9^4 + 4^4 + 7^4 + 4^4)
35 print(check_armstrong(0)) # Armstrong Number (boundary)
36 print(check_armstrong(1)) # Armstrong Number (boundary)
37 print(check_armstrong(-153)) # Invalid Input (boundary)
38 print(check_armstrong("abc")) # Invalid Input (invalid type)
```

Output :



```
/usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task3.py
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.py/t
ask3.py
Armstrong Number
Armstrong Number
Not an Armstrong Number
Armstrong Number
Armstrong Number
Armstrong Number
Invalid Input
Invalid Input
syedsufiyan@Syeds-MacBook-Air-3 3.1.py %
```

Observation:

The program correctly checks whether a given number is an Armstrong number by calculating the sum of each digit raised to the power of the total number of digits. The few-shot examples help guide the logic and ensure accurate identification of Armstrong and non-Armstrong numbers. The implementation properly handles edge cases such as 0 and 1, which are valid Armstrong numbers, and includes input validation to reject negative values and non-integer inputs. Test results confirm that

the function behaves correctly across normal, boundary, and invalid cases, demonstrating robustness and correctness.

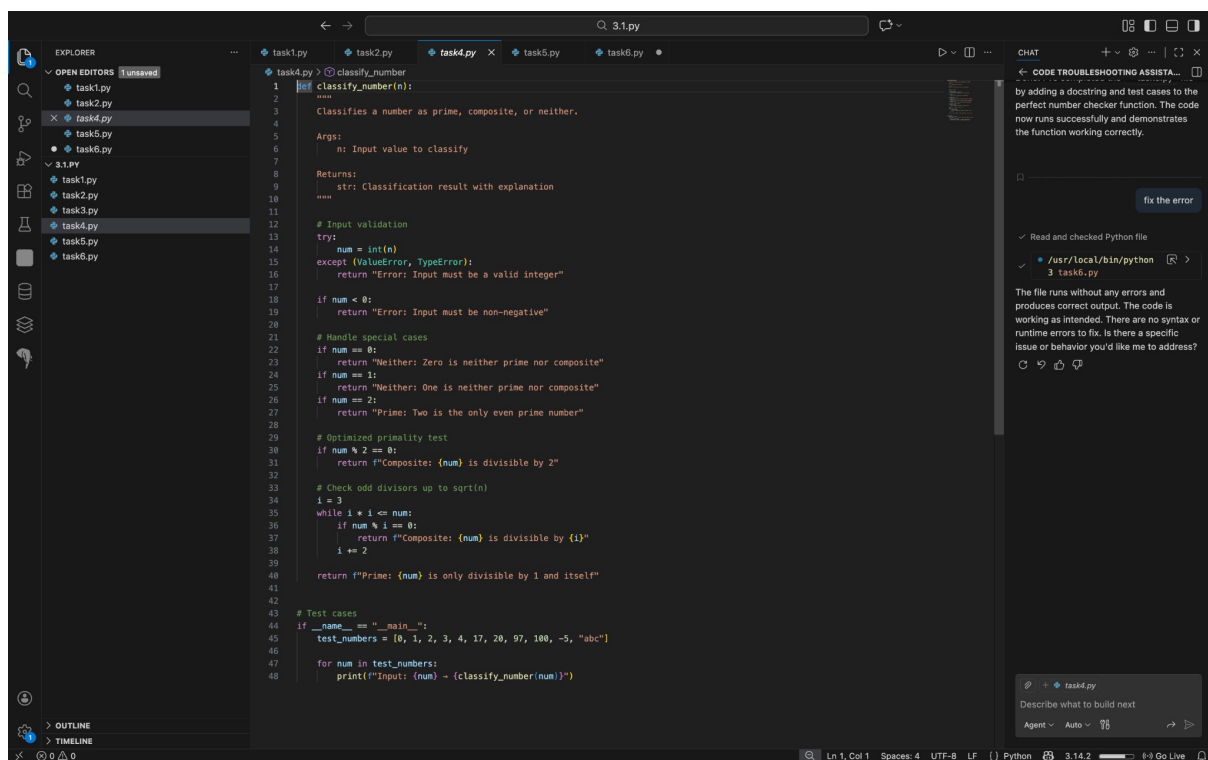
Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.

Code :



```
1 def classify_number(n):
2     """
3     Classifies a number as prime, composite, or neither.
4
5     Args:
6         n: Input value to classify
7
8     Returns:
9         str: Classification result with explanation
10    """
11
12    # Input validation
13    try:
14        num = int(n)
15    except (ValueError, TypeError):
16        return "Error: Input must be a valid integer"
17
18    if num < 0:
19        return "Error: Input must be non-negative"
20
21    # Handle special cases
22    if num == 0:
23        return "Neither: Zero is neither prime nor composite"
24    if num == 1:
25        return "Neither: One is neither prime nor composite"
26    if num == 2:
27        return "Prime: Two is the only even prime number"
28
29    # Optimized primality test
30    if num % 2 == 0:
31        return f"Composite: {num} is divisible by 2"
32
33    # Check odd divisors up to sqrt(n)
34    i = 3
35    while i * i <= num:
36        if num % i == 0:
37            return f"Composite: {num} is divisible by {i}"
38        i += 2
39
40    return f"Prime: {num} is only divisible by 1 and itself"
41
42
43 # Test cases
44 if __name__ == "__main__":
45     test_numbers = [0, 1, 2, 3, 4, 17, 20, 97, 100, -5, "abc"]
46
47     for num in test_numbers:
48         print(f"Input: {num} -> {classify_number(num)}")
```

CHAT

CODE TROUBLESHOOTING ASSISTANT

by adding a docstring and test cases to the perfect number checker function. The code now runs successfully and demonstrates the function working correctly.

fix the error

✓ Read and checked Python file

✓ /usr/local/bin/python 3 task6.py

The file runs without any errors and produces correct output. The code is working as intended. There are no syntax or runtime errors to fix. Is there a specific issue or behavior you'd like me to address?

+ task4.py

Describe what to build next

Agent Auto

Output :

```
/usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task4.py
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.py/ta
Input: 0 → Neither: Zero is neither prime nor composite
Input: 1 → Neither: One is neither prime nor composite
Input: 2 → Prime: Two is the only even prime number
Input: 3 → Prime: 3 is only divisible by 1 and itself
Input: 4 → Composite: 4 is divisible by 2
Input: 17 → Prime: 17 is only divisible by 1 and itself
Input: 20 → Composite: 20 is divisible by 2
Input: 97 → Prime: 97 is only divisible by 1 and itself
Input: 100 → Composite: 100 is divisible by 2
Input: -5 → Error: Input must be non-negative
Input: abc → Error: Input must be a valid integer
syedsufiyan@Syeds-MacBook-Air-3 3.1.py %
```

Observation:

The program accurately classifies numbers as prime, composite, or neither by applying proper input validation and an optimized primality test. It correctly handles special cases such as 0 and 1, identifies 2 as the only even prime number, and efficiently checks divisibility for other numbers up to their square root. Invalid and negative inputs are handled gracefully with meaningful error messages. The test results confirm that the program produces correct and explanatory classifications for a wide range of inputs.

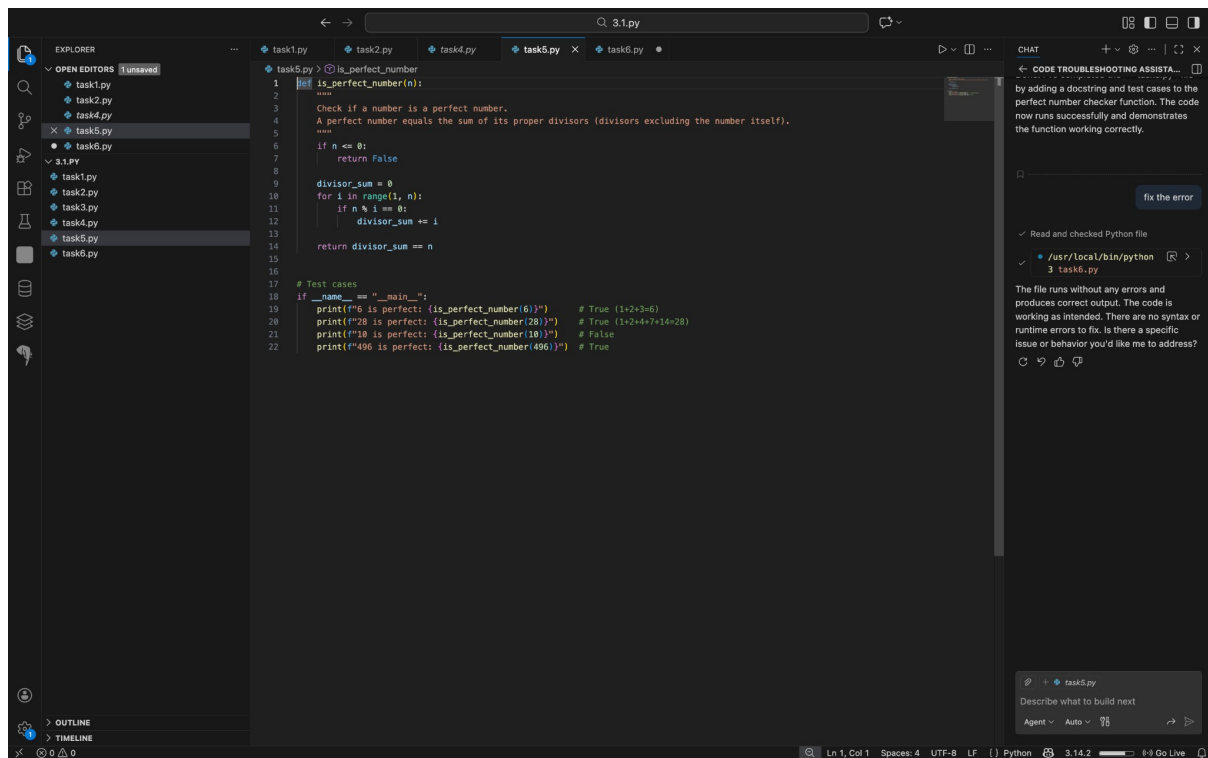
Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic

Code :



Output :

```
/usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task5.py
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task5.py
6 is perfect: True
28 is perfect: True
10 is perfect: False
496 is perfect: True
syedsufiyan@Syeds-MacBook-Air-3 3.1.py %
```

Observation:

The program correctly determines whether a given number is a perfect number by calculating the sum of its proper divisors and comparing it with the original number. It properly excludes non-positive values and accurately identifies known perfect numbers such as 6, 28, and 496, while correctly rejecting non-perfect numbers like 10. The test cases confirm that the divisor-based logic works as expected, demonstrating correct use of loops and conditional statements.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output

examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

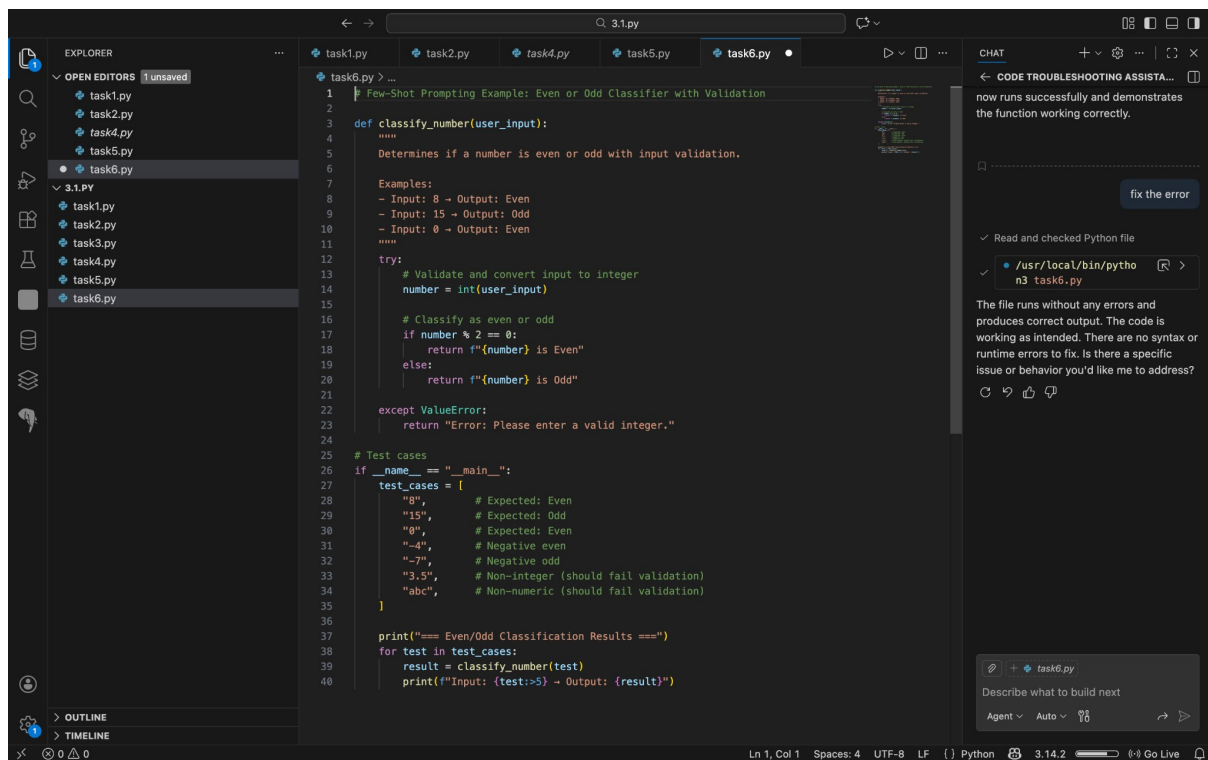
Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

Code :



```
1 Few-Shot Prompting Example: Even or Odd Classifier with Validation
2
3
4 def classify_number(user_input):
5     """
6     Determines if a number is even or odd with input validation.
7
8     Examples:
9     - Input: 8 → Output: Even
10    - Input: 15 → Output: Odd
11    - Input: 0 → Output: Even
12    """
13    try:
14        # Validate and convert input to integer
15        number = int(user_input)
16
17        # Classify as even or odd
18        if number % 2 == 0:
19            return f"{number} is Even"
20        else:
21            return f"{number} is Odd"
22    except ValueError:
23        return "Error: Please enter a valid integer."
24
25 # Test cases
26 if __name__ == "__main__":
27     test_cases = [
28         "8", # Expected: Even
29         "15", # Expected: Odd
30         "0", # Expected: Even
31         "-4", # Negative even
32         "-7", # Negative odd
33         "3.5", # Non-integer (should fail validation)
34         "abc", # Non-numeric (should fail validation)
35     ]
36
37     print("=== Even/Odd Classification Results ===")
38     for test in test_cases:
39         result = classify_number(test)
40         print(f"Input: {test} - Output: {result}")
```

CHAT

CODE TROUBLESHOOTING ASSISTANT

now runs successfully and demonstrates the function working correctly.

fix the error

✓ Read and checked Python file

✓ `/usr/local/bin/python3 task6.py`

The file runs without any errors and produces correct output. The code is working as intended. There are no syntax or runtime errors to fix. Is there a specific issue or behavior you'd like me to address?

Describe what to build next

Agent Auto

Output :

```
/usr/local/bin/python3 /Users/syedsufiyan/3.1.py/task6.py
syedsufiyan@Syeds-MacBook-Air-3 3.1.py % /usr/local/bin/python3 /Users/syedsufiyan/3.1.py/t
ask6.py
=== Even/Odd Classification Results ===
Input:      8 → Output: 8 is Even
Input:     15 → Output: 15 is Odd
Input:      0 → Output: 0 is Even
Input:     -4 → Output: -4 is Even
Input:     -7 → Output: -7 is Odd
Input:    3.5 → Output: Error: Please enter a valid integer.
Input:    abc → Output: Error: Please enter a valid integer.
syedsufiyan@Syeds-MacBook-Air-3 3.1.py %
```

Observation :

The program correctly identifies even and odd numbers using few-shot prompting while validating user input. It handles positive numbers, negative numbers, and zero accurately. Invalid inputs such as floating-point values and non-numeric strings are safely rejected using exception handling, ensuring robust and error-free execution.